CS519 Locks Bench: Performance and perf Analysis

Saroop

1 Benchmark Overview

The benchmark creates `num_threads` threads. Each thread repeatedly: (1) acquires a lock, (2) increments a selected counter inside the critical section for `work_length` steps, and (3) releases the lock. Total runtime is reported as "Total time ... seconds".

Lock types:

- 1: Spin lock (Compare-and-Swap, CAS)
- 2: Spin lock (Test-and-Set, TAS)
- 3: Spin lock (Ticket)
- 4: Mutex
- 5: Semaphore

2 Experimental Setup

- Iterations = 100000

Two sweeps were run:

- **Thread sweep (Table 1):** vary threads, fix work_length = 10.
- **Work-length sweep (Table 2):** vary work_length, fix threads = 16.

3 Results

3.1 Table 1: Total time vs threads (work_length = 10)

Threads	CAS (1)	TAS (2)	Ticket (3)	Mutex (4)	Semaphore (5)
1	0.007082	0.006336	0.007275	0.005269	0.004907
2	0.044943	0.022157	0.089262	0.030771	0.088153
4	0.238187	0.210110	0.206969	0.050551	0.152117
8	0.534091	0.547241	0.467003	0.207463	0.362018
16	1.487978	1.333226	1.250182	0.569441	0.769984

Table 1: Execution time (seconds) as threads increase (iterations=100000, work_length=10).

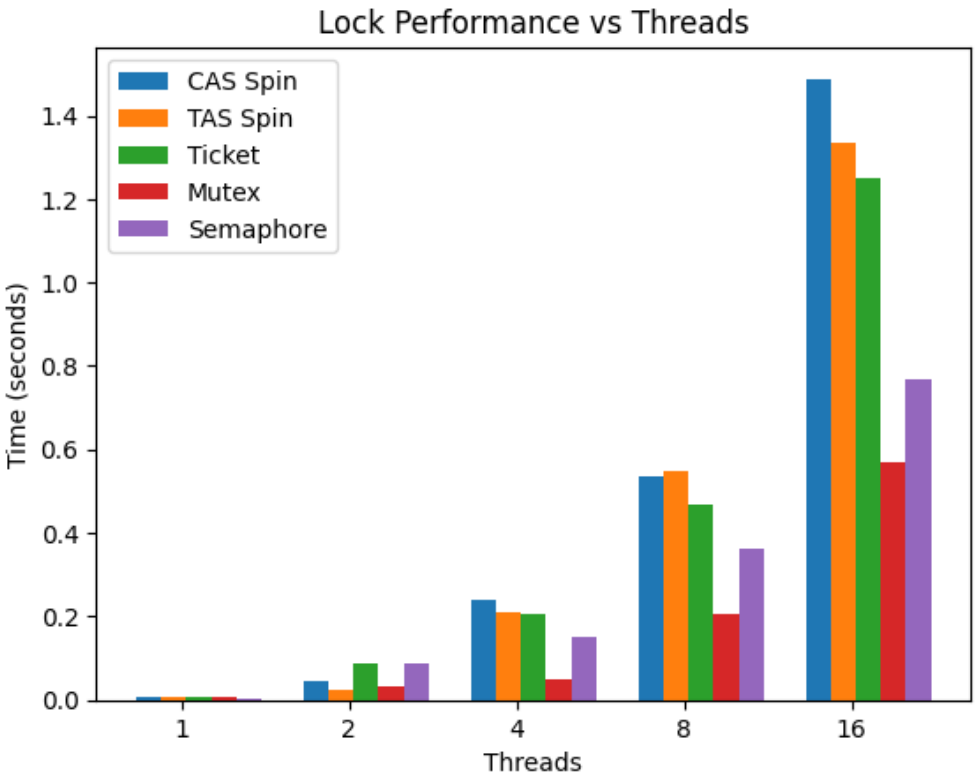
3.2 Table 2: Total time vs work length (threads = 16)

Work length	CAS (1)	TAS (2)	Ticket (3)	Mutex (4)	Semaphore (5)
1	1.448787	1.326927	1.225602	0.411297	0.702560
10	1.538046	1.234847	1.371302	0.553973	0.739537
100	2.164495	2.011440	2.431239	1.459517	1.425784
1000	6.522159	6.620569	13.268761	20.391199	20.464034

Table 2: Execution time as critical-section work increases (threads=16, iterations=100000).

3.3 Graphs

Figure 1: Grouped bar chart of execution time vs thread count (work_length=10, iterations=100000).



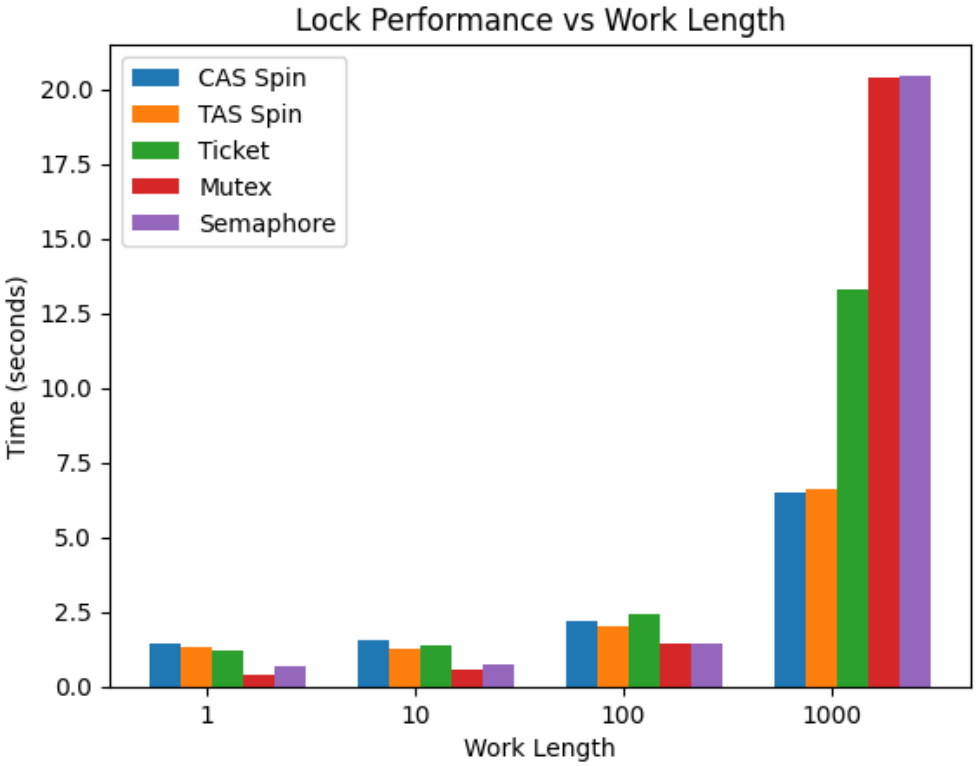


Figure 2: Grouped bar chart of execution time vs work length (threads=16, iterations=100000).

4 perf Analysis

`perf record` samples CPU activity and attributes it to functions and call stacks.

- For spin locks: CPU time is expected to concentrate in lock acquisition and spinning.
- For blocking locks: CPU time is expected to involve kernel paths (futex wait/wake and scheduling).

4.1 Dominant functions by lock type (16 threads, work_length=10)

This section summarizes the key dominant functions observed from perf reports.

Lock 1: CAS spinlock Perf analysis shows that the vast majority of CPU cycles are spent in getting the lock. Approximately 93 percent of total cycles are attributed to `acquire_lock`, with about 92 percent of self time concentrated in `caslock_lock`. This indicates heavy busy-wait spinning under contention, where threads repeatedly attempt atomic CAS operations in userspace. Only a small fraction of cycles, around 5 percent, are spent in `do_work`, showing that the workload inside the critical section is minor compared to the cost of acquiring the lock.

Lock 2: TAS spinlock Similar to CAS: cycles concentrate in `acquire_lock` and `taslock_lock`. TAS generally causes heavy cache-line bouncing under contention because every failed attempt writes, so the lock variable becomes highly contended.

Lock 3: Ticket lock Perf analysis shows that a large fraction of execution time is spent in kernel scheduling paths rather than purely in userspace spinning. While `acquire_lock` still dominates the high-level call path, a significant portion of cycles are attributed to `__sched_yield` and the associated syscall and scheduler functions, including `entry_SYSCALL_64`, `do_sched_yield`, and `schedule`. This indicates that the ticket lock implementation yields the CPU under contention instead of continuously busy-waiting. As a result, overhead shifts from userspace spinning to kernel-mediated context switching and scheduling decisions. Only a small fraction of cycles are spent in `ticket_acquire` itself and in `do_work`, showing that performance is dominated by yielding and scheduler activity rather than useful work inside the critical section. This behavior contrasts with CAS and TAS spinlocks and explains the high kernel cycle counts observed for the ticket lock.

Lock 4: Mutex Perf analysis shows that execution time is dominated by kernel activity rather than userspace spinning. A significant portion of cycles are spent in futex-related paths, including `__x64_sys_futex`, `futex_wait`, and `futex_wake`, as well as scheduler functions such as `schedule` and `__schedule`. This indicates that contended mutex acquisitions cause threads to block and sleep in the kernel instead of busy-waiting. Additionally, a large fraction of cycles appear in CPU idle paths (for example `cpuidle_enter` and `mwait_idle`), showing that CPUs are often idle while threads are waiting on the mutex. Compared to spin locks, relatively little time is spent in `acquire_lock`, and a substantial fraction of execution time is devoted to `do_work`, reflecting reduced contention overhead and effective use of blocking synchronization.

Lock 5: Semaphore Perf analysis shows that execution time is largely dominated by kernel activity and CPU idle time. A substantial fraction of cycles are spent in CPU idle paths such as `cpuidle_enter`, `cpuidle_enter_state`, and `mwait_idle`, indicating that threads frequently block and CPUs remain idle while waiting on the semaphore. Semaphore operations involve futex-based blocking, as evidenced by significant time in `__x64_sys_futex`, `futex_wait`, and related scheduling functions. The presence of `sem_post` in the profile reflects kernel-mediated wakeups when permits are released. Compared to spin locks, relatively little time is spent attempting to acquire the lock, and a noticeable fraction of execution time is spent in `do_work`, showing that semaphore-based synchronization avoids busy-waiting by putting threads to sleep and relying on the OS scheduler to manage contention.

4.2 Userspace vs OS split using cycles:u and cycles:k

To quantify where cycles execute, the benchmark was profiled twice per lock:

- `perf record -e cycles:u ...` to count cycles in userspace
- `perf record -e cycles:k ...` to count cycles in the kernel

Then `perf report --stdio --no-children --sort dso` was used to capture event counts.

4.2.1 Event counts (approx.) and fractions (16 threads, iterations=100000, work_length=10)

Lock	cycles:u	cycles:k	Userspace %	Kernel %
1 (CAS)	55,418,881,972	55,787,561	99.90%	0.10%
2 (TAS)	49,555,361,478	54,757,734	99.89%	0.11%
3 (Ticket)	3,445,393,118	46,803,732,674	6.86%	93.14%
4 (Mutex)	1,261,225,039	13,700,209,222	8.43%	91.57%
5 (Semaphore)	1,735,767,856	21,741,778,503	7.39%	92.61%

Table 3: Userspace vs kernel cycle counts using `cycles:u` and `cycles:k`

4.2.2 Interpretation

- **Locks 1 and 2 (CAS, TAS):** overwhelmingly userspace cycles, which is consistent with busy-wait spinning.
- **Locks 3, 4, 5 (Ticket, Mutex, Semaphore):** overwhelmingly kernel cycles in this dataset, meaning the implementation frequently enters the kernel (yield, futex wait/wake, scheduling). This is consistent with blocking or yielding behavior under contention.

5 Discussion

5.1 Highest overhead by configuration

As threads increase (Table 1) At higher thread counts, contention increases. Spin locks (CAS and TAS) waste CPU time spinning, so runtime rises quickly. Mutex and semaphore can reduce spinning by blocking, and in this dataset mutex is the fastest at 16 threads.

As work_length increases (Table 2) When the critical section does more work, total time rises for every lock. At very large work (work_length=1000), total runtime becomes dominated by time spent inside the critical section, and lock management overhead becomes less important relative to the work itself.

5.2 Where time is spent

- **Spin locks:** time concentrates in `acquire_lock` and the lock-specific spin routine (`caslock_lock` or `taslock_lock`). This is mostly userspace execution.
- **Ticket, mutex, semaphore:** time shifts toward kernel mechanisms (yield, futex wait/wake, scheduler). This shows up as large `cycles:k` counts and kernel functions dominating reports.

6 Commands Used

- Thread sweep (Table 1): `./locks_bench <lock> <threads> 100000 10`
- Work-length sweep (Table 2): `./locks_bench <lock> 16 100000 <work_length>`
- Dominant functions (call stacks): `perf record -g ./locks_bench <lock> 16 100000 10` then `perf report -g "graph,0.5,caller"`

- Userspace vs kernel split (repeat per lock):
  - `perf record -e cycles:u ./locks_bench <lock> 16 100000 10`
  - `perf report --stdio --no-children --sort dso`
  - `perf record -e cycles:k ./locks_bench <lock> 16 100000 10`
  - `perf report --stdio --no-children --sort dso`